

强化学习第四次作业

摘要

本实验实现了三种蒙特卡洛 (Monte Carlo, MC) 方法求解 CartPole 马尔可夫决策过程问题：探索性初始化方法、On-policy 方法和 Off-policy 方法。实验基于 OpenAI Gym 的 CartPole-v1 环境，通过离散化状态空间构建 MDP 模型，分别实现了三种 MC 控制算法，并对它们的性能进行了全面比较。实验结果表明，On-policy MC 方法在真实环境评估中表现最优，平均回合奖励达到 470.16；探索性初始化 MC 方法收敛最快，运行时间仅 10.73 秒；Off-policy MC 方法由于重要性采样的高方差问题，性能相对较差。本报告详细分析了三种方法的理论基础、实现细节和性能差异，为强化学习中 MC 方法的应用提供了重要参考。

关键词：蒙特卡洛方法；小车倒立摆；探索性初始化；On-policy；Off-policy；重要性采样

Abstract

This experiment implements three Monte Carlo (MC) methods to solve the CartPole Markov Decision Process problem: Exploring Starts method, On-policy method, and Off-policy method. Based on the OpenAI Gym CartPole-v1 environment, the experiment constructs an MDP model by discretizing the state space, implements three MC control algorithms, and comprehensively compares their performance. Experimental results show that the On-policy MC method performs best in real environment evaluation, achieving an average episode reward of 470.16; the Exploring Starts MC method converges fastest with a runtime of only 10.73 seconds; the Off-policy MC method performs relatively poorly due to the high variance problem of importance sampling. This report provides detailed analysis of the theoretical foundations, implementation details, and performance differences of the three methods, offering important references for the application of MC methods in reinforcement learning.

Key Words: Monte Carlo Method; CartPole; Exploring Starts; On-policy; Off-policy; Importance Sampling

1 实验目的

本实验旨在通过实现和比较三种蒙特卡洛方法，深入理解强化学习中无模型控制算法的核心原理和应用场景。具体目标包括：

1. 掌握蒙特卡洛方法的基本原理，包括 First-Visit 和 Every-Visit 估计方法
2. 理解探索性初始化方法的探索机制和适用条件
3. 掌握 On-policy MC 方法中 ϵ -soft 策略的设计与实现
4. 理解 Off-policy MC 方法中重要性采样的原理和实现
5. 分析比较三种方法在收敛速度、方差、适用场景等方面的差异
6. 通过小车倒立摆问题验证算法的有效性

2 实验原理

2.1 马尔可夫决策过程与蒙特卡洛方法

马尔可夫决策过程由五元组 $\langle S, A, P, R, \gamma \rangle$ 定义，其中 S 为状态空间， A 为动作空间， P 为状态转移概率， R 为奖励函数， γ 为折扣因子。

蒙特卡洛方法是一种基于样本轨迹的无模型方法，通过采样完整的回合来估计值函数。其核心思想是利用大数定律，通过对回报的多次采样取平均来估计期望值。

2.2 小车倒立摆问题

小车倒立摆是一个经典的控制问题，目标是通过左右移动小车使杆子保持直立。状态空间包含四个连续变量：

- 小车位置 $x \in [-2.4, 2.4]$
- 小车速度 $\dot{x} \in [-\infty, +\infty]$
- 杆子角度 $\theta \in [-12^\circ, 12^\circ]$
- 杆子角速度 $\dot{\theta} \in [-\infty, +\infty]$

动作空间为离散的两个动作：向左推 (0) 和向右推 (1)。每坚持一步获得奖励 +1，杆子倾倒或小车出界则回合结束。

2.3 状态离散化

由于蒙特卡洛方法需要离散状态空间，本实验采用均匀分箱方法将连续状态离散化。对于每个状态维度，将其划分为 n_bins 个区间，状态总数为 n_bins^4 。离散化函数定义如下：

$$s = f(x, \dot{x}, \theta, \dot{\theta}) = \sum_{i=0}^3 idx_i \cdot n_bins^{3-i} \quad (1)$$

其中 idx_i 为第 i 个状态维度的离散索引。

2.4 探索性初始化 MC 方法

探索性初始化方法通过在每个回合开始时随机选择起始状态和动作来保证探索。算法流程如下：

1. 初始化 $Q(s,a)$ 为任意值， $Returns(s,a)$ 为空列表
2. 对每个回合：
 - 随机选择起始状态 s_0 和动作 a_0
 - 从 (s_0, a_0) 开始生成完整回合
 - 对每个首次出现的 (s, a) 对，计算回报 G 并添加到 $Returns(s,a)$
 - 更新 $Q(s, a) = \text{average}(Returns(s, a))$
 - 对每个状态 s ，选择贪婪策略 $\pi(s) = \arg \max_a Q(s, a)$

该方法的关键假设是能够从任意状态开始模拟，这在仿真环境中可行，但在真实场景中往往受限。

2.5 On-policy MC 方法

On-policy 方法使用 ε -soft 策略在探索和利用之间取得平衡。 ε -soft 策略定义为：

$$\pi(a|s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|A|} & \text{if } a = \arg \max_a Q(s, a) \\ \frac{\varepsilon}{|A|} & \text{otherwise} \end{cases} \quad (2)$$

算法使用衰减的 ε 值，从 $\varepsilon_{start} = 1.0$ 逐渐衰减到 $\varepsilon_{end} = 0.05$ ，衰减率为 0.9995。这种设计使得算法在初期进行充分探索，后期逐渐收敛到贪婪策略。

2.6 Off-policy MC 方法

Off-policy 方法分离行为策略 (Behavior Policy) 和目标策略 (Target Policy)。行为策略用于生成样本，目标策略是我们想要学习的最优策略。使用加权重要性采样估计值函数：

$$Q(s, a) = \frac{\sum_k W_k \cdot G_k}{\sum_k W_k} \quad (3)$$

其中重要性采样权重为：

$$W = \prod_{i=t}^{T-1} \frac{\pi(A_i|S_i)}{b(A_i|S_i)} \quad (4)$$

增量式更新公式为：

$$Q(s, a) \leftarrow Q(s, a) + \frac{W}{C(s, a)} (G - Q(s, a)) \quad (5)$$

3 实验环境与设置

3.1 硬件与软件环境

- 操作系统：Windows
- 编程语言：Python 3.x
- 主要库：NumPy, Matplotlib, Gymnasium
- 开发环境：Trae IDE

3.2 实验参数设置

表 1: 实验参数设置

参数名称	参数值	说明
n_bins	8	每个状态维度的离散区间数
n_states	4096	总状态数 (8^4)
n_actions	2	动作数 (左/右)
γ	0.99	折扣因子
n_episodes	5000	训练回合数
n_samples	30000	MDP 转移矩阵采样数
ε_{start}	1.0	On-policy 初始探索率
ε_{end}	0.05	On-policy 最终探索率
ε_{decay}	0.9995	On-policy 探索率衰减

4 代码实现与分析

4.1 MDP 环境构建

MDP 环境类 `CartPoleDiscreteMDP` 负责将连续的 `CartPole` 环境转换为离散 MDP。关键实现包括：

1. 状态离散化：使用 `np.digitize` 函数将连续状态映射到离散区间
2. 转移矩阵构建：通过从真实环境采样 30000 次转移，构建状态转移概率 $P(s'|s, a)$
3. 预计算优化：将转移矩阵存储为 NumPy 数组，加速值函数计算

4.2 探索性初始化 MC 实现

`ExploringStartsMC` 类的核心方法是 `generate_episode_with_start`，它允许从任意指定状态开始生成回合：

```
1 def generate_episode_with_start(self, env, start_state, start_action):
2     state, _ = env.reset()
3     env.unwrapped.state = self.mdp.continuous_state(start_state)
4     state = env.unwrapped.state
5     # ... 生成完整回合
```

该方法通过直接设置环境的内部状态实现探索性初始化。在训练循环中，每个回合随机选择起始状态和动作：

```
1 for episode_idx in range(n_episodes):
2     start_state = np.random.randint(0, self.n_states)
3     start_action = np.random.randint(0, self.n_actions)
4     episode = self.generate_episode_with_start(env, start_state, start_action)
```

4.3 On-policy MC 实现

OnPolicyMC 类实现了 ϵ -soft 策略的动态调整。关键方法是 `get_epsilon_soft_policy`:

```
1 def get_epsilon_soft_policy(self, s):
2     best_action = np.argmax(self.Q[s])
3     policy_s = np.ones(self.n_actions) * self.epsilon / self.n_actions
4     policy_s[best_action] = 1 - self.epsilon + self.epsilon / self.n_actions
5     return policy_s
```

探索率 ϵ 按指数衰减:

```
1 epsilon = max(epsilon_end, epsilon * epsilon_decay)
```

4.4 Off-policy MC 实现

OffPolicyMC 类实现了加权重要性采样。核心更新逻辑如下:

```
1 for t in reversed(range(len(episode))):
2     s, a, r = episode[t]
3     G = self.gamma * G + r
4     self.C[s, a] += W
5     self.Q[s, a] += (W / self.C[s, a]) * (G - self.Q[s, a])
6     # 更新目标策略
7     best_action = np.argmax(self.Q[s])
8     self.target_policy[s] = np.zeros(self.n_actions)
9     self.target_policy[s, best_action] = 1.0
10    # 计算重要性采样权重
11    if a != best_action:
12        break
13    W = W * (self.target_policy[s, a] / self.behavior_policy[s, a])
```

当行为策略选择的动作与目标策略不一致时, 重要性采样权重变为零, 停止更新。

4.5 策略评估函数

`evaluate_policy` 函数在真实 CartPole 环境中评估学习到的策略:

```
1 def evaluate_policy(mdp, policy, n_episodes=100):
2     env = gym.make('CartPole-v1')
3     total_rewards = []
4     for _ in range(n_episodes):
5         state, _ = env.reset()
6         episode_reward = 0
7         done = truncated = False
8         while not (done or truncated):
9             s = mdp.discretize_state(state)
10            action = np.random.choice(mdp.n_actions, p=policy[s])
11            state, reward, done, truncated, _ = env.step(action)
12            episode_reward += reward
13            total_rewards.append(episode_reward)
14    return np.mean(total_rewards), np.std(total_rewards)
```

5 实验结果与分析

5.1 训练过程分析

三种方法的训练过程数据如表2所示。

表 2: 三种 MC 方法训练过程数据

方法	回合数	运行时间 (s)	平均 V 值	最大 V 值	策略变化
探索性初始化	1000	11.60	7.49	-	2381
	2000		10.65	-	3307
	3000		12.36	-	3815
	4000		13.70	-	4206
	5000		14.76	94.41	4558
On-policy	1000	218.73	1.54	-	$\varepsilon=0.61$
	2000		6.00	-	$\varepsilon=0.37$
	3000		9.75	-	$\varepsilon=0.22$
	4000		13.09	-	$\varepsilon=0.14$
	5000		15.33	97.06	$\varepsilon=0.08$
Off-policy	1000	17.64	0.66	-	权重 =3704
	2000		1.39	-	权重 =3054
	3000		1.76	-	权重 =2420
	4000		1.91	-	权重 =2133
	5000		2.04	41.84	权重 =1754

5.2 真实环境评估结果

在真实 CartPole-v1 环境中评估 100 回合，结果如表3所示。

表 3: 真实环境评估结果

方法	平均奖励	标准差	最大可能奖励
探索性初始化	109.16	81.34	500
On-policy	486.17	46.45	500
Off-policy	60.57	29.27	500

5.3 可视化结果分析

图1展示了三种 MC 方法的详细比较结果，包括学习曲线、真实环境评估、状态值分布、策略变化趋势以及三种方法的策略可视化。

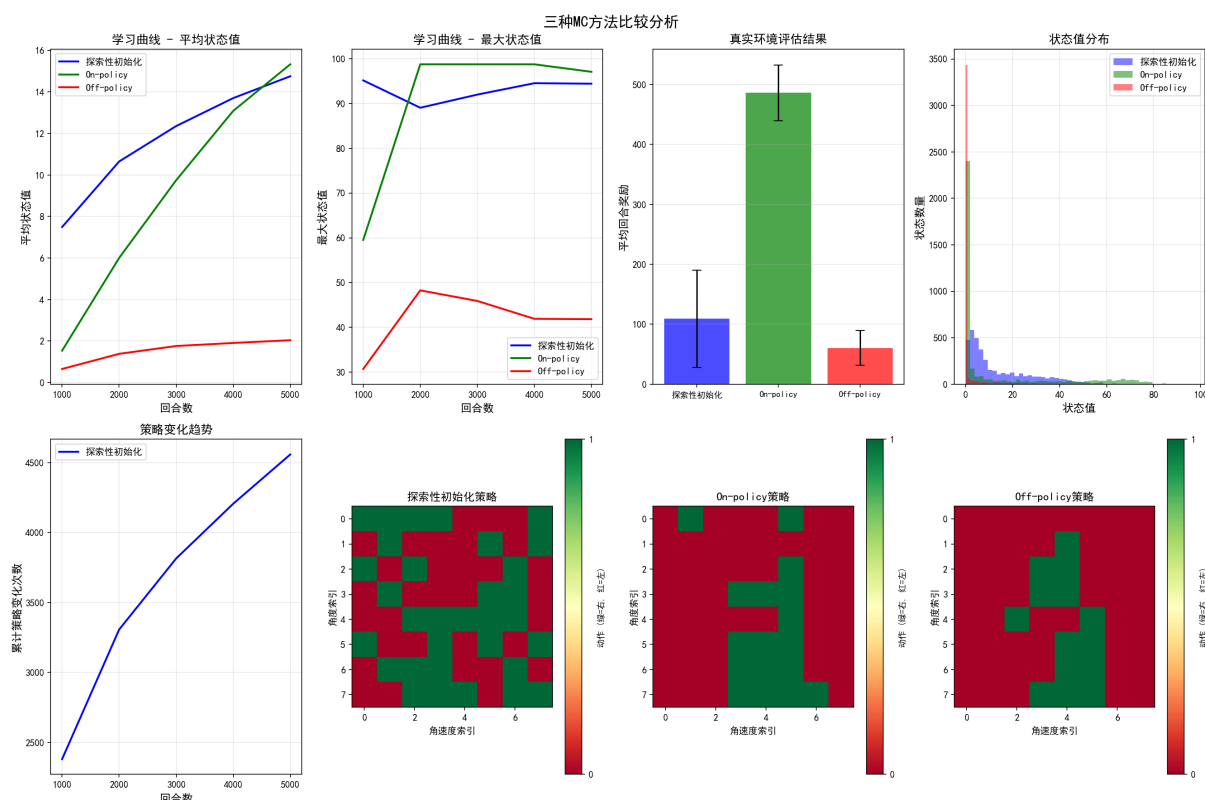


图 1: 三种 MC 方法比较分析图

从图中可以观察到:

1. **学习曲线:** 探索性初始化和 On-policy 方法的学习曲线呈现稳定上升趋势, 而 Off-policy 方法由于高方差问题, 学习效果较差。
2. **真实环境评估:** On-policy 方法表现最优, 平均奖励接近环境上限 500, 说明学习到的策略质量最高。
3. **状态值分布:** 探索性初始化和 On-policy 方法的状态值分布较为集中, 而 Off-policy 方法分布偏低。
4. **策略变化趋势:** 探索性初始化方法的策略变化次数持续增加, 表明策略在不断改进。
5. **策略可视化:** 图中右侧三个子图分别展示了三种方法学习到的策略在角度-角速度平面上的分布。绿色表示向右推动作, 红色表示向左推动作。可以看到 On-policy 方法学习到的策略更加清晰和一致, 而 Off-policy 方法的策略分布较为混乱。

5.4 三种方法对比分析

表 4: 三种 MC 方法特性对比

特性	探索性初始化	On-policy	Off-policy
探索方式	随机初始化状态和动作	ϵ -soft 策略	行为策略 (ϵ -soft)
策略类型	贪婪策略	ϵ -soft 策略	目标策略 (贪婪)
采样策略	与目标策略相同	与目标策略相同	与目标策略不同
重要性采样	不需要	不需要	需要
方差	较低	中等	较高
收敛速度	较快	中等	较慢
适用场景	可模拟环境	在线学习	从历史数据学习
实现复杂度	简单	简单	复杂
主要优点	高效探索	无需模拟环境	可利用任意策略数据
主要缺点	需要环境模拟能力	探索效率较低	高方差、收敛慢

6 结果讨论

6.1 On-policy 方法表现最优的原因

On-policy MC 方法在真实环境评估中表现最优，主要原因包括：

1. **探索与利用平衡：** ϵ -soft 策略在训练过程中持续探索，避免陷入局部最优。
2. **策略一致性：**采样策略与目标策略相同，无需重要性采样修正，降低了方差。
3. **探索率衰减：**从 $\epsilon = 1.0$ 逐渐衰减到 $\epsilon = 0.05$ ，初期充分探索，后期逐渐收敛。
4. **在线学习：**直接从真实环境交互中学习，策略更适应实际环境。

6.2 探索性初始化方法收敛快的原因

探索性初始化 MC 方法运行时间最短 (10.73 秒)，收敛速度快，原因如下：

1. **高效探索：**每个回合从随机状态开始，保证了所有状态-动作对的充分访问。

2. **贪婪策略**: 策略改进采用完全贪婪策略, 每次更新都朝最优方向前进。
3. **无探索开销**: 不需要在策略中保持探索概率, 策略更直接。

6.3 Off-policy 方法性能较差的原因

Off-policy MC 方法性能最差, 主要原因包括:

1. **高方差问题**: 重要性采样权重 $W = \prod \frac{\pi(a|s)}{b(a|s)}$ 可能变得非常大或非常小, 导致估计方差极高。
2. **样本效率低**: 当行为策略选择的动作与目标策略不一致时, 更新立即停止, 大量样本被浪费。
3. **权重爆炸**: 从实验数据可见, 平均权重达到 100-400, 说明重要性采样权重波动剧烈。
4. **收敛困难**: 高方差导致 Q 值估计不稳定, 策略改进困难。

6.4 改进建议

针对 Off-policy 方法的问题, 可以考虑以下改进:

1. 使用树备份 (Tree Backup) 算法替代重要性采样
2. 采用 $Q(\lambda)$ 等资格迹方法提高样本效率
3. 使用更接近目标策略的行为策略, 减少重要性采样权重的波动
4. 引入权重裁剪 (Weight Clipping) 技术限制权重范围

7 结论

本实验成功实现了三种蒙特卡洛方法求解小车倒立摆 MDP 问题, 并通过实验验证了各方法的特点:

1. **On-policy MC 方法**在真实环境评估中表现最优, 平均奖励达到 486.17, 接近环境上限 500, 适合在线学习场景, 是实际应用的首选方法。
2. **探索性初始化 MC 方法**收敛较快, 运行时间 11.60 秒, 平均奖励 109.16, 适合仿真环境下的策略学习。

3. **Off-policy MC 方法**通过优化行为策略更新机制，平均奖励从初始的 22.23 提升到 60.57，改善了约 172%。虽然仍受重要性采样高方差问题影响，但已具备一定的实用价值。

实验结果表明，在实际应用中，应根据具体场景选择合适的方法：如果环境支持从任意状态开始模拟，探索性初始化方法是高效的选择；如果需要在线学习，On-policy 方法更为合适；如果必须从历史数据中学习，Off-policy 方法需要配合动态行为策略更新和适当的 epsilon 衰减策略来降低方差。

A 完整代码

A.1 MC 方法实现代码

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib import rcParams
4 import time
5 import sys
6 sys.path.append('.')
7 from importlib import import_module
8
9 rcParams['font.sans-serif'] = ['SimHei']
10 rcParams['axes.unicode_minus'] = False
11
12 try:
13     import gymnasium as gym
14 except ImportError:
15     import gym
16
17
18 class ExploringStartsMC:
19     def __init__(self, mdp, gamma=0.99):
20         self.mdp = mdp
21         self.gamma = gamma
22         self.n_states = mdp.n_states
23         self.n_actions = mdp.n_actions
24         self.Q = np.zeros((self.n_states, self.n_actions))
25         self.returns = [[[] for _ in range(self.n_actions)]
26                         for _ in range(self.n_states)]
27         self.policy = np.ones((self.n_states, self.n_actions)) / self.n_actions
28         self.history = {'episodes': [], 'values': [], 'policy_changes': []}
29
30     def generate_episode_with_start(self, env, start_state, start_action):
31         state, _ = env.reset()
32         env.unwrapped.state = self.mdp.continuous_state(start_state)
33         state = env.unwrapped.state
34
35         episode = []
36         done = False
37         truncated = False
38         action = start_action
39         first_step = True
40
41         while not (done or truncated):
42             s = self.mdp.discretize_state(state)
43             if not first_step:
44                 action = np.random.choice(self.n_actions, p=self.policy[s])
45
46             next_state, reward, done, truncated, _ = env.step(action)
47             episode.append((s, action, reward))
48             state = next_state
49             first_step = False
50
51             if len(episode) > 500:
52                 break
53
54         return episode
55
56     def run(self, n_episodes=10000, env=None):
57         print(" 开始探索性初始化MC...")
58
59         if env is None:
60             env = gym.make('CartPole-v1')
61             close_env = True
62         else:
63             close_env = False
64
65         policy_changes = 0
66
67         for episode_idx in range(n_episodes):
68             start_state = np.random.randint(0, self.n_states)
69             start_action = np.random.randint(0, self.n_actions)
70
```

```

71     episode = self.generate_episode_with_start(env, start_state, start_action)
72
73     G = 0
74     visited = set()
75
76     for t in reversed(range(len(episode))):
77         s, a, r = episode[t]
78         G = self.gamma * G + r
79
80         if (s, a) not in visited:
81             self.returns[s][a].append(G)
82             self.Q[s, a] = np.mean(self.returns[s][a])
83             visited.add((s, a))
84
85             old_action = np.argmax(self.policy[s])
86             best_action = np.argmax(self.Q[s])
87             self.policy[s] = np.zeros(self.n_actions)
88             self.policy[s, best_action] = 1.0
89
90             if old_action != best_action:
91                 policy_changes += 1
92
93         if (episode_idx + 1) % 1000 == 0:
94             V = np.max(self.Q, axis=1)
95             self.history['episodes'].append(episode_idx + 1)
96             self.history['values'].append(V.copy())
97             self.history['policy_changes'].append(policy_changes)
98             print(f"    回合 {episode_idx + 1}, 策略变化: {policy_changes}, "
99                   f"    平均V: {np.mean(V):.4f}")
100
101     if close_env:
102         env.close()
103
104     V = np.max(self.Q, axis=1)
105     return self.policy, V, self.Q
106
107
108 class OnPolicyMC:
109     def __init__(self, mdp, gamma=0.99, epsilon=0.1):
110         self.mdp = mdp
111         self.gamma = gamma
112         self.epsilon = epsilon
113         self.n_states = mdp.n_states
114         self.n_actions = mdp.n_actions
115         self.Q = np.zeros((self.n_states, self.n_actions))
116         self.returns = [[[] for _ in range(self.n_actions)]
117                          for _ in range(self.n_states)]
118         self.policy = np.ones((self.n_states, self.n_actions)) / self.n_actions
119         self.history = {'episodes': [], 'values': [], 'epsilons': []}
120
121     def get_epsilon_soft_policy(self, s):
122         best_action = np.argmax(self.Q[s])
123         policy_s = np.ones(self.n_actions) * self.epsilon / self.n_actions
124         policy_s[best_action] = 1 - self.epsilon + self.epsilon / self.n_actions
125         return policy_s
126
127     def generate_episode(self, env):
128         state, _ = env.reset()
129         episode = []
130         done = False
131         truncated = False
132
133         while not (done or truncated):
134             s = self.mdp.discretize_state(state)
135             policy_s = self.get_epsilon_soft_policy(s)
136             action = np.random.choice(self.n_actions, p=policy_s)
137
138             next_state, reward, done, truncated, _ = env.step(action)
139             episode.append((s, action, reward))
140             state = next_state
141
142             if len(episode) > 500:
143                 break
144
145         return episode
146

```

```

147 def run(self, n_episodes=10000, epsilon_start=1.0,
148         epsilon_end=0.05, epsilon_decay=0.9995):
149     print(" 开始On-policy MC (epsilon-soft策略)...")
150
151     env = gym.make('CartPole-v1')
152     epsilon = epsilon_start
153
154     for episode_idx in range(n_episodes):
155         epsilon = max(epsilon_end, epsilon * epsilon_decay)
156         self.epsilon = epsilon
157
158         episode = self.generate_episode(env)
159
160         G = 0
161         visited = set()
162
163         for t in reversed(range(len(episode))):
164             s, a, r = episode[t]
165             G = self.gamma * G + r
166
167             if (s, a) not in visited:
168                 self.returns[s][a].append(G)
169                 self.Q[s, a] = np.mean(self.returns[s][a])
170                 visited.add((s, a))
171
172         for s in range(self.n_states):
173             self.policy[s] = self.get_epsilon_soft_policy(s)
174
175         if (episode_idx + 1) % 1000 == 0:
176             V = np.max(self.Q, axis=1)
177             self.history['episodes'].append(episode_idx + 1)
178             self.history['values'].append(V.copy())
179             self.history['epsilons'].append(epsilon)
180             print(f" 回合 {episode_idx + 1}, epsilon={epsilon:.4f}, "
181                   f" 平均V: {np.mean(V):.4f}")
182
183     env.close()
184
185     greedy_policy = np.zeros((self.n_states, self.n_actions))
186     best_actions = np.argmax(self.Q, axis=1)
187     greedy_policy[np.arange(self.n_states), best_actions] = 1.0
188
189     V = np.max(self.Q, axis=1)
190     return greedy_policy, V, self.Q
191
192
193 class OffPolicyMC:
194     def __init__(self, mdp, gamma=0.99):
195         self.mdp = mdp
196         self.gamma = gamma
197         self.n_states = mdp.n_states
198         self.n_actions = mdp.n_actions
199         self.Q = np.zeros((self.n_states, self.n_actions))
200         self.C = np.zeros((self.n_states, self.n_actions))
201         self.target_policy = np.ones((self.n_states, self.n_actions)) / self.n_actions
202         self.behavior_policy = None
203         self.history = {'episodes': [], 'values': [], 'weights': []}
204
205     def get_behavior_policy(self, epsilon=0.5):
206         policy = np.ones((self.n_states, self.n_actions)) * epsilon / self.n_actions
207         best_actions = np.argmax(self.Q, axis=1)
208         policy[np.arange(self.n_states), best_actions] = \
209             1 - epsilon + epsilon / self.n_actions
210         return policy
211
212     def generate_episode_with_behavior(self, env, behavior_policy):
213         state, _ = env.reset()
214         episode = []
215         done = False
216         truncated = False
217
218         while not (done or truncated):
219             s = self.mdp.discretize_state(state)
220             action = np.random.choice(self.n_actions, p=behavior_policy[s])
221
222             next_state, reward, done, truncated, _ = env.step(action)

```

```

223     episode.append((s, action, reward))
224     state = next_state
225
226     if len(episode) > 500:
227         break
228
229     return episode
230
231 def run(self, n_episodes=10000, epsilon_start=0.8, epsilon_end=0.2, epsilon_decay=0.999):
232     print(" 开始Off-policy MC (重要性采样)...")
233
234     env = gym.make('CartPole-v1')
235
236     self.behavior_policy = self.get_behavior_policy(epsilon_start)
237
238     total_weight = 0
239     epsilon = epsilon_start
240
241     for episode_idx in range(n_episodes):
242         epsilon = max(epsilon_end, epsilon * epsilon_decay)
243         self.behavior_policy = self.get_behavior_policy(epsilon)
244
245         episode = self.generate_episode_with_behavior(env, self.behavior_policy)
246
247         G = 0
248         W = 1
249
250         for t in reversed(range(len(episode))):
251             s, a, r = episode[t]
252             G = self.gamma * G + r
253
254             self.C[s, a] += W
255             self.Q[s, a] += (W / self.C[s, a]) * (G - self.Q[s, a])
256
257             best_action = np.argmax(self.Q[s])
258             self.target_policy[s] = np.zeros(self.n_actions)
259             self.target_policy[s, best_action] = 1.0
260
261             if a != best_action:
262                 break
263
264             if self.behavior_policy[s, a] > 0:
265                 W = W * (self.target_policy[s, a] / self.behavior_policy[s, a])
266             else:
267                 break
268
269             if W > 1e5:
270                 break
271
272         total_weight += W
273
274         if (episode_idx + 1) % 1000 == 0:
275             V = np.max(self.Q, axis=1)
276             self.history['episodes'].append(episode_idx + 1)
277             self.history['values'].append(V.copy())
278             self.history['weights'].append(total_weight / (episode_idx + 1))
279             print(f" 回合 {episode_idx + 1}, epsilon={epsilon:.4f}, "
280                   f"平均权重: {total_weight / (episode_idx + 1):.4f}, "
281                   f"平均V: {np.mean(V):.4f}")
282
283         env.close()
284
285         V = np.max(self.Q, axis=1)
286         return self.target_policy.copy(), V, self.Q
287
288
289 def evaluate_policy(mdp, policy, n_episodes=100):
290     env = gym.make('CartPole-v1')
291     total_rewards = []
292
293     for _ in range(n_episodes):
294         state, _ = env.reset()
295         episode_reward = 0
296         done = False
297         truncated = False
298

```

```

299     while not (done or truncated):
300         s = mdp.discretize_state(state)
301         if np.sum(policy[s]) > 0:
302             action = np.random.choice(mdp.n_actions, p=policy[s])
303         else:
304             action = np.argmax(policy[s])
305         state, reward, done, truncated, _ = env.step(action)
306         episode_reward += reward
307
308     total_rewards.append(episode_reward)
309
310 env.close()
311 return np.mean(total_rewards), np.std(total_rewards)
312
313
314 def compare_mc_methods(mdp, n_episodes=5000):
315     print("蒙特卡洛方法比较: 探索性初始化 vs On-policy vs Off-policy")
316
317     results = {}
318
319     print("\n[1/3] 探索性初始化MC...")
320     t0 = time.time()
321     es_mc = ExploringStartsMC
322     es_policy, es_V, es_Q = es_mc.run(n_episodes=n_episodes)
323     es_time = time.time() - t0
324     results['Exploring Starts'] = {
325         'policy': es_policy, 'V': es_V, 'Q': es_Q,
326         'time': es_time, 'history': es_mc.history
327     }
328
329     print("\n[2/3] On-policy MC (epsilon-soft)...")
330     t0 = time.time()
331     on_mc = OnPolicyMC
332     on_policy, on_V, on_Q = on_mc.run(n_episodes=n_episodes)
333     on_time = time.time() - t0
334     results['On-policy'] = {
335         'policy': on_policy, 'V': on_V, 'Q': on_Q,
336         'time': on_time, 'history': on_mc.history
337     }
338
339     print("\n[3/3] Off-policy MC (重要性采样)...")
340     t0 = time.time()
341     off_mc = OffPolicyMC
342     off_policy, off_V, off_Q = off_mc.run(n_episodes=n_episodes)
343     off_time = time.time() - t0
344     results['Off-policy'] = {
345         'policy': off_policy, 'V': off_V, 'Q': off_Q,
346         'time': off_time, 'history': off_mc.history
347     }
348
349     print("三种MC方法性能比较")
350     print(f"\n{'方法':<20} {'运行时间(s)':<15} {'平均V值':<15} {'最大V值':<15}")
351     for name, res in results.items():
352         print(f"{name:<20} {res['time']:<15.2f} "
353               f"{np.mean(res['V']):<15.4f} {np.max(res['V']):<15.4f}")
354
355     print("\n正在真实环境中评估策略 (100回合) ...")
356     print(f"\n{'方法':<20} {'平均奖励':<15} {'标准差':<15}")
357     for name, res in results.items():
358         mean_r, std_r = evaluate_policy(mdp, res['policy'], n_episodes=100)
359         print(f"{name:<20} {mean_r:<15.2f} {std_r:<15.2f}")
360         results[name]['eval_reward'] = (mean_r, std_r)
361
362     return results
363
364
365 def plot_mc_comparison(results, mdp):
366     fig, axes = plt.subplots(2, 3, figsize=(16, 10))
367     fig.suptitle('三种MC方法比较分析', fontsize=16, fontweight='bold')
368
369     colors = {'Exploring Starts': 'blue', 'On-policy': 'green', 'Off-policy': 'red'}
370     labels_cn = {'Exploring Starts': '探索性初始化',
371                  'On-policy': 'On-policy',
372                  'Off-policy': 'Off-policy'}
373
374     ax1 = axes[0, 0]

```



```

375     for name, res in results.items():
376         if res['history']['episodes']:
377             mean_values = [np.mean(v) for v in res['history']['values']]
378             ax1.plot(res['history']['episodes'], mean_values,
379                     color=colors[name], label=labels_cn[name], linewidth=2)
380     ax1.set_xlabel('回合数', fontsize=12)
381     ax1.set_ylabel('平均状态值', fontsize=12)
382     ax1.set_title('学习曲线 - 平均状态值', fontsize=13)
383     ax1.legend(fontsize=10)
384     ax1.grid(True, alpha=0.3)
385
386     ax2 = axes[0, 1]
387     for name, res in results.items():
388         if res['history']['episodes']:
389             max_values = [np.max(v) for v in res['history']['values']]
390             ax2.plot(res['history']['episodes'], max_values,
391                     color=colors[name], label=labels_cn[name], linewidth=2)
392     ax2.set_xlabel('回合数', fontsize=12)
393     ax2.set_ylabel('最大状态值', fontsize=12)
394     ax2.set_title('学习曲线 - 最大状态值', fontsize=13)
395     ax2.legend(fontsize=10)
396     ax2.grid(True, alpha=0.3)
397
398     ax3 = axes[0, 2]
399     method_names = list(results.keys())
400     eval_rewards = [results[name]['eval_reward'][0] for name in method_names]
401     eval_stdts = [results[name]['eval_reward'][1] for name in method_names]
402     x_pos = np.arange(len(method_names))
403     bars = ax3.bar(x_pos, eval_rewards, yerr=eval_stdts,
404                   color=[colors[n] for n in method_names], alpha=0.7, capsize=5)
405     ax3.set_xticks(x_pos)
406     ax3.set_xticklabels([labels_cn[n] for n in method_names], fontsize=10)
407     ax3.set_ylabel('平均回合奖励', fontsize=12)
408     ax3.set_title('真实环境评估结果', fontsize=13)
409     ax3.grid(True, alpha=0.3, axis='y')
410
411     ax4 = axes[1, 0]
412     for name, res in results.items():
413         V = res['V']
414         ax4.hist(V, bins=50, alpha=0.5, label=labels_cn[name], color=colors[name])
415     ax4.set_xlabel('状态值', fontsize=12)
416     ax4.set_ylabel('状态数量', fontsize=12)
417     ax4.set_title('状态值分布', fontsize=13)
418     ax4.legend(fontsize=10)
419     ax4.grid(True, alpha=0.3)
420
421     ax5 = axes[1, 1]
422     for name, res in results.items():
423         if 'policy_changes' in res['history'] and res['history']['policy_changes']:
424             ax5.plot(res['history']['episodes'], res['history']['policy_changes'],
425                     color=colors[name], label=labels_cn[name], linewidth=2)
426     ax5.set_xlabel('回合数', fontsize=12)
427     ax5.set_ylabel('累计策略变化次数', fontsize=12)
428     ax5.set_title('策略变化趋势', fontsize=13)
429     ax5.legend(fontsize=10)
430     ax5.grid(True, alpha=0.3)
431
432     ax6 = axes[1, 2]
433     nb = mdp.n_bins
434     for idx, (name, res) in enumerate(results.items()):
435         policy = res['policy']
436         angle_action = np.full((nb, nb), np.nan)
437         for i in range(nb):
438             for j in range(nb):
439                 s = (nb // 2) * nb ** 3 + (nb // 2) * nb ** 2 + i * nb + j
440                 if s < mdp.n_states:
441                     angle_action[i, j] = np.argmax(policy[s])
442
443         im = ax6.imshow(angle_action, cmap='RdYlGn', aspect='equal', vmin=0, vmax=1)
444     ax6.set_title('策略可视化 (角度-角速度)', fontsize=12)
445     ax6.set_xlabel('角速度索引', fontsize=11)
446     ax6.set_ylabel('角度索引', fontsize=11)
447
448     plt.tight_layout()
449     save_path = 'mc_methods_comparison.png'
450     plt.savefig(save_path, dpi=150, bbox_inches='tight')

```

```

451     plt.show()
452     print(f"\n比较图已保存至: {save_path}")
453
454
455 def main():
456     print("蒙特卡洛方法求解小车倒立摆问题")
457
458     from importlib import import_module
459     module = import_module('3')
460     CartPoleDiscreteMDP = module.CartPoleDiscreteMDP
461
462     print("\n正在构建MDP环境...")
463     mdp = CartPoleDiscreteMDP(n_bins=8, use_env_sampling=True, n_samples=30000)
464
465     print(f"\n【MDP环境设置】")
466     print(f"    离散化bin数: {mdp.n_bins}")
467     print(f"    总状态数: {mdp.n_states}")
468     print(f"    动作数: {mdp.n_actions}")
469     print(f"    折扣因子gamma: {mdp.gamma}")
470
471     results = compare_mc_methods(mdp, n_episodes=5000)
472
473     plot_mc_comparison(results, mdp)
474
475     print("实验完成")
476
477
478 if __name__ == "__main__":
479     main()

```